Data and Computer Communications

Data Link Control (DLC)

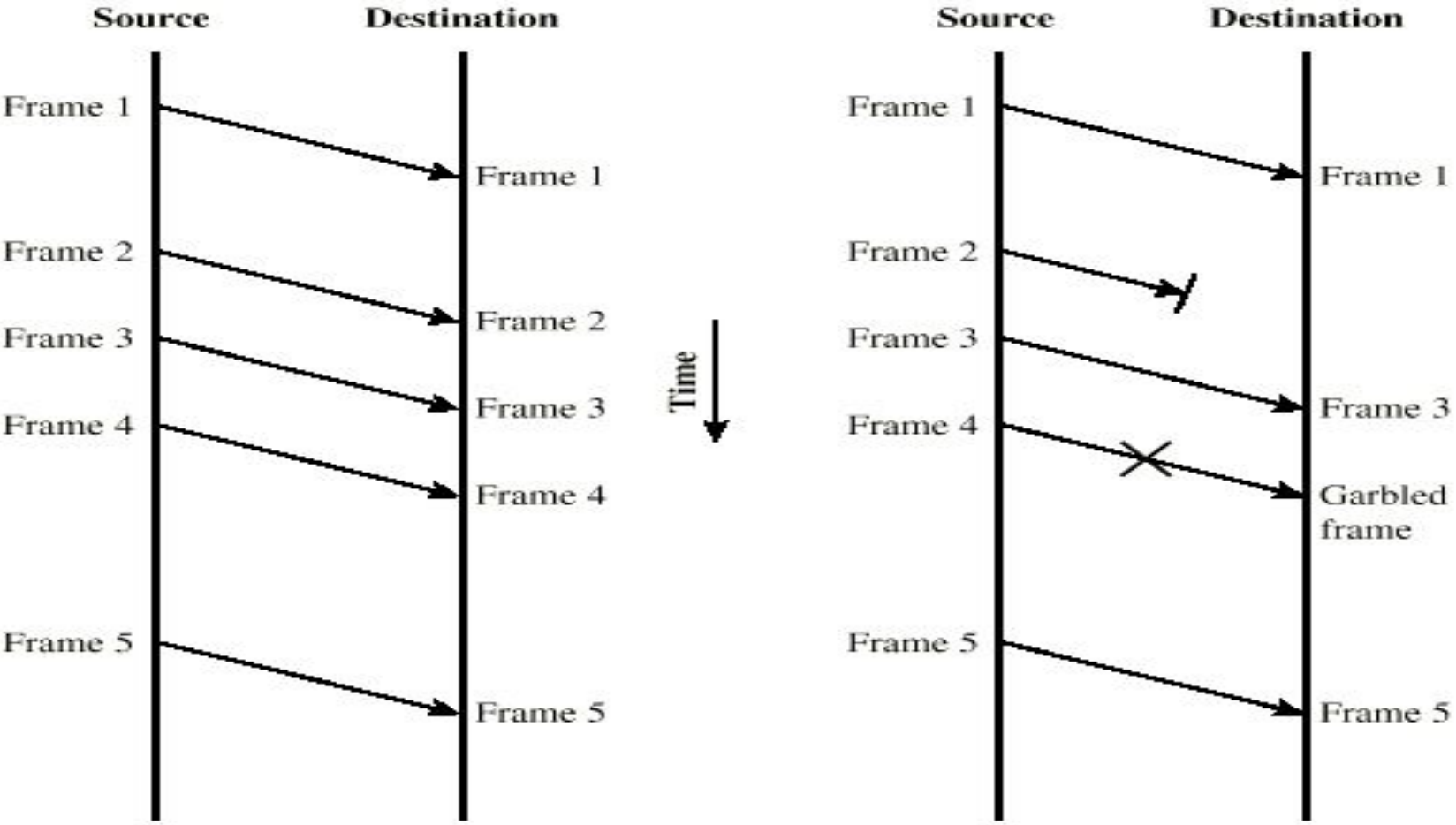
Data Link Layer i.e. L2

- Service provider to L3
- DLL service is split into two categories
  - Logical Link Control (LLC) <= upper sublayer
    - Data link control (Simplex protocols)
    - Deals with link management
      - Framing, flow control, error control, acknowledgement service etc
  - Medium Access Control (MAC) <= Lower sublayer
    - two forms of MAC: distributed and centralized
    - who is allowed to access the media at any one time
    - ALOHA, CSMA/CD, CSMA/CA etc

Flow Control

- Ensuring the sending entity does not overwhelm the receiving entity
  - Preventing buffer overflow
- Transmission time
  - Time taken to emit all bits into medium
- Propagation time
  - Time for a bit to traverse the link

Model of Frame Transmission



(a) Error-free transmission

(b) Transmission with losses and errors

Stop and Wait Flow Control

- The simplest form of flow control is stop and wait
- Source transmits frame
- Destination receives frame and replies with acknowledgement
- **Source waits for ACK before sending next frame**
- **Destination can stop flow by not sending ACK**
- Works well for a few large frames, but not good for large block of data or frames

Fragmentation

- Large block of data may be split into small frames by the source. This is done because of :
  - Limited buffer size
  - Errors detection is sooner (when whole frame received)
  - On error, retransmission of smaller frames is needed
  - Prevents one station occupying medium for long periods

- Stop and wait becomes inadequate, with the use of multiple frames for a single message
  - the problem is that only one frame at a time can be in transit.
    - Because it implements simplex communication

Issues of Stop and Wait

- For very **high data rates**; for very **long distances** between sender and receiver, stop-and-wait flow control provides **inefficient line utilization.**
- Suppose the transmission time is 1 unit of time depending on source terminal.
- If the (propagation time >1) the line is always underutilized and even

  for (propagation time <1) the line is inefficiently utilized.

Issues of Stop and Wait

- The essence of the problem is that only one frame at a time can be in transit. To explain we first define the bit length of a link as follows:

$$B=R*d/V$$

Where, B= length of a link in bits, i.e. the no. of bits present on a link

V=velocity of propagation, in m/s,

d=length, or distance, of the link in meters,

R=data rate of the link, in bps

Then Propagation delay/time a is:

$$a = B/L$$

Where, L= is the number of bits in the frame (length of the frame in bits).

Stop and Wait Link Utilization

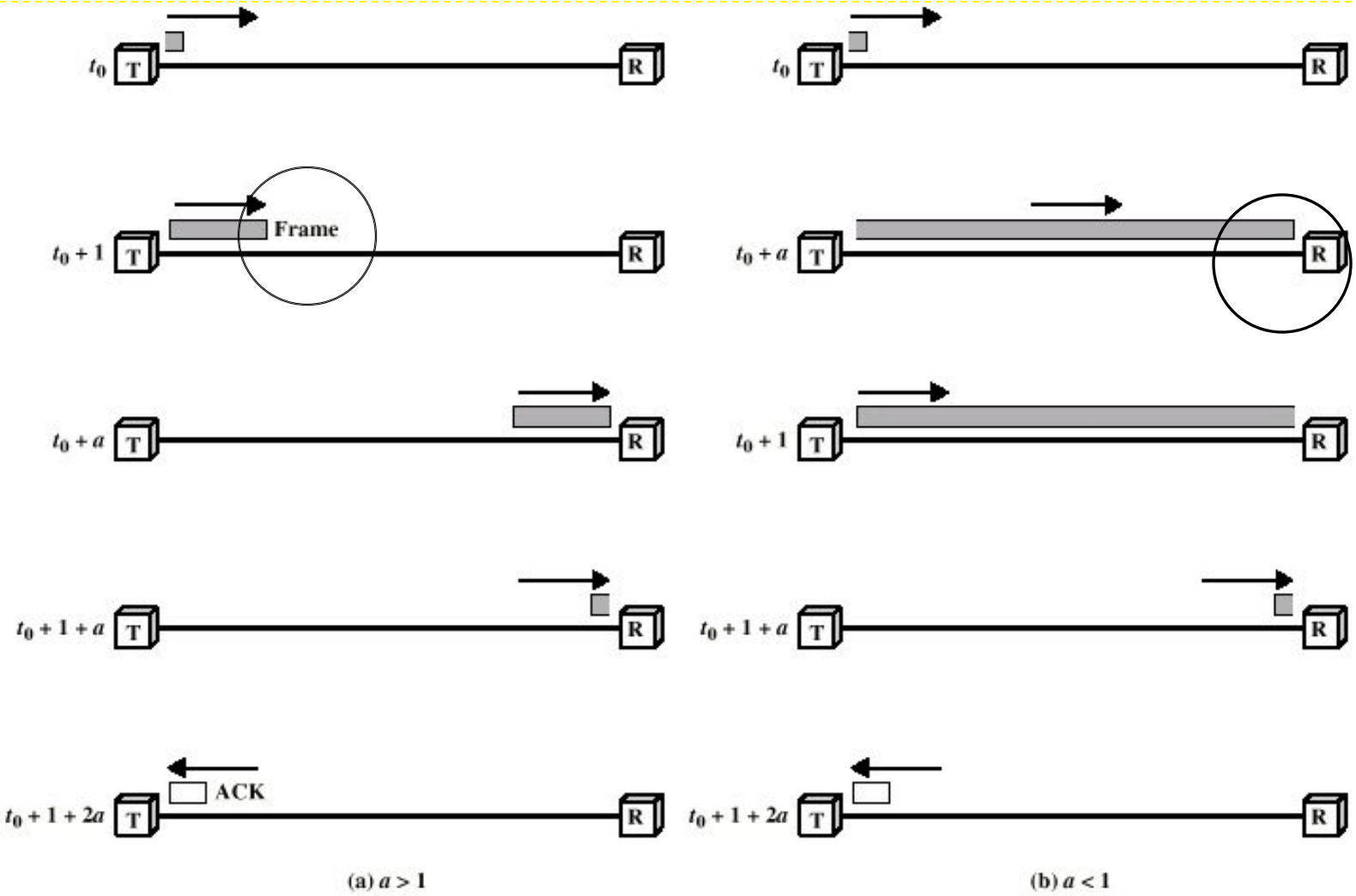


Figure 7.2 Stop-and-Wait Link Utilization (transmission time = 1; propagation time = a)

Stop and Wait Link Utilization

- When $a > 1$, the propagation time is greater (i.e. Slow) than the transmission time.
  - In this case, the sender completes transmission of the entire frame before the leading bits of that frame arrive at the receiver.
  - Therefore, larger values of *a* are consistent with higher data rates and/or longer distances between stations.
    - **Underutilization**
- When $a < 1$, the propagation time (i.e. Fast) is less than the transmission time.
  - In this case, the frame is sufficiently long that the first bits of the frame have arrived at the destination before the source has completed the transmission of the frame.
    - **Inefficient utilization**
- Therefore, **only one frame at a time can be in transit**. And in situations where the **bit length of the link is greater than the frame length** serious inefficiencies result.
- Efficiency can be greatly improved by **allowing multiple frames to be in transit** at the same time.

Sliding Windows Flow Control

- Allow multiple frames to be in transit
- Receiver has buffer space of W long
- Transmitter can send up to W frames without ACK
- Each frame is labeled with sequence number, to keep track of which frames are ACKed.
- ACK includes number of next frame expected
- Sequence number bounded by size of field (k)
  - Frames are numbered modulo 2^k

Sliding Windows Flow Control

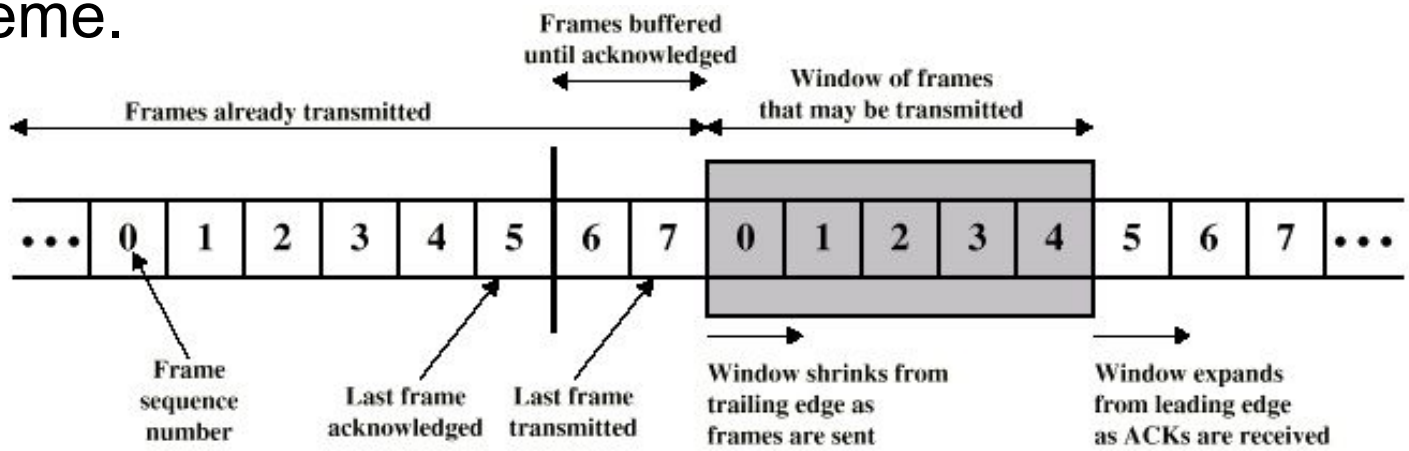
- Say for two stations, A (sender) and B (receiver), connected via a full-duplex link implements this scheme.
- B ACKs a frame by sending an ACK that includes the Seq. No. of the next frame expected. This ACK also implicitly announces that B is prepared to receive the next *W* frames, beginning with the number specified.
- This scheme can also be used to ACK multiple frames considering them to be as *window* of frames.

Sliding Windows Flow Control

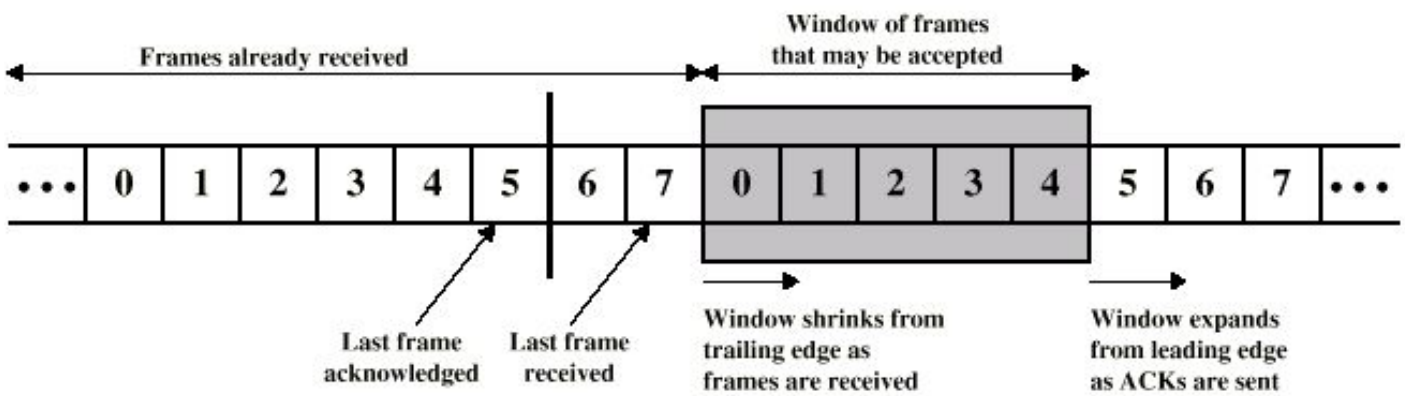
- B could receive frames 2, 3, and 4 but withhold ACK until frame 4 has arrived. By then returning an ACK with Seq. No. 5, B acknowledges frames 2, 3, and 4 at one time.
- A maintains a list of sequence numbers that it is allowed to send, and B maintains a list of sequence numbers that it is prepared to receive.
- Each of these lists can be thought of as a *window* of frames. The operation is referred to as **sliding-window flow control**.
  - Frames are numbered modulo 2^k

Sliding Window Diagram

- It considers to use a 3-bit sequence number to implement the scheme.

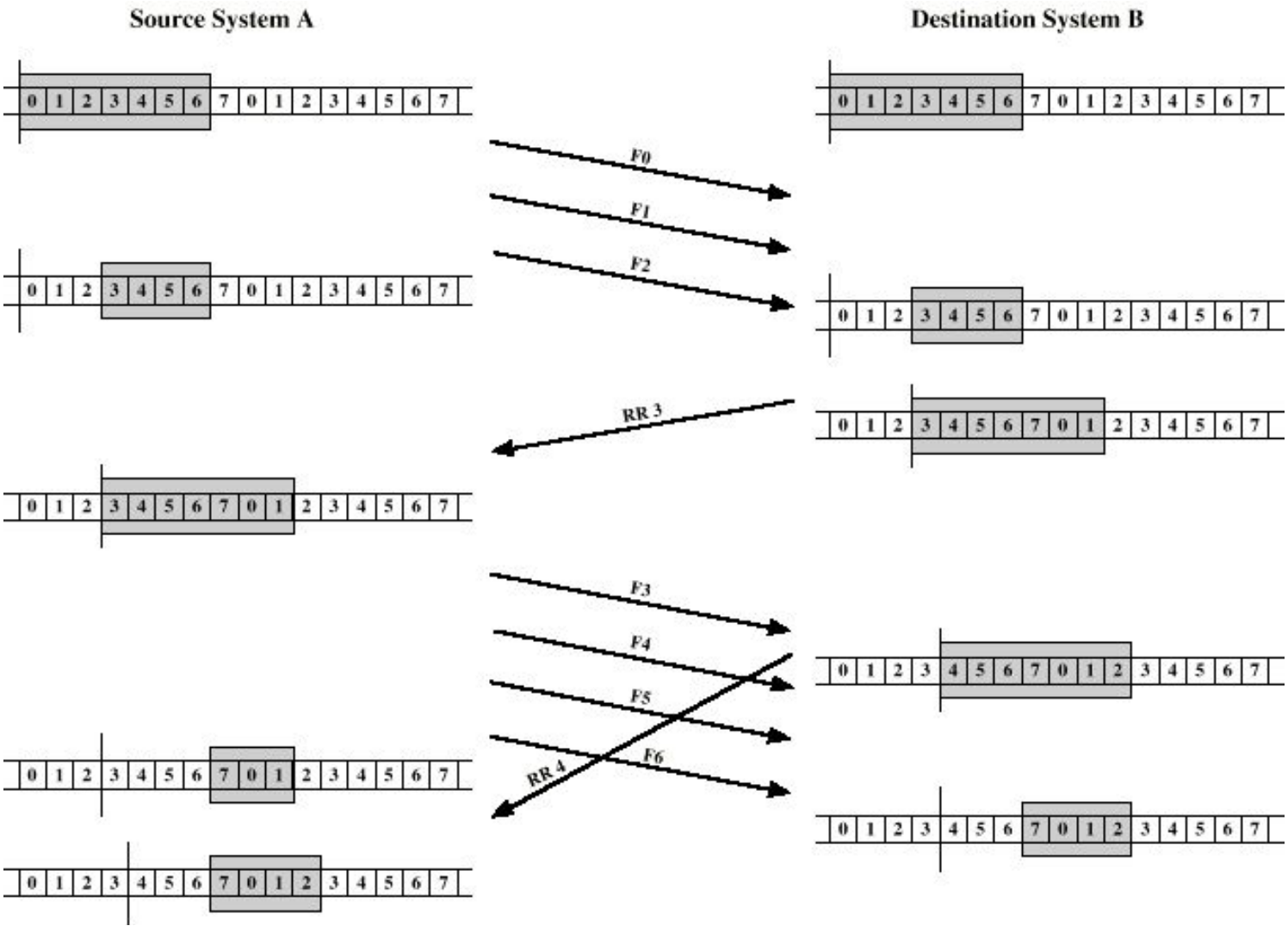


(a) Sender's perspective



(b) Receiver's perspective

Example Sliding Window



Sliding Window Enhancements

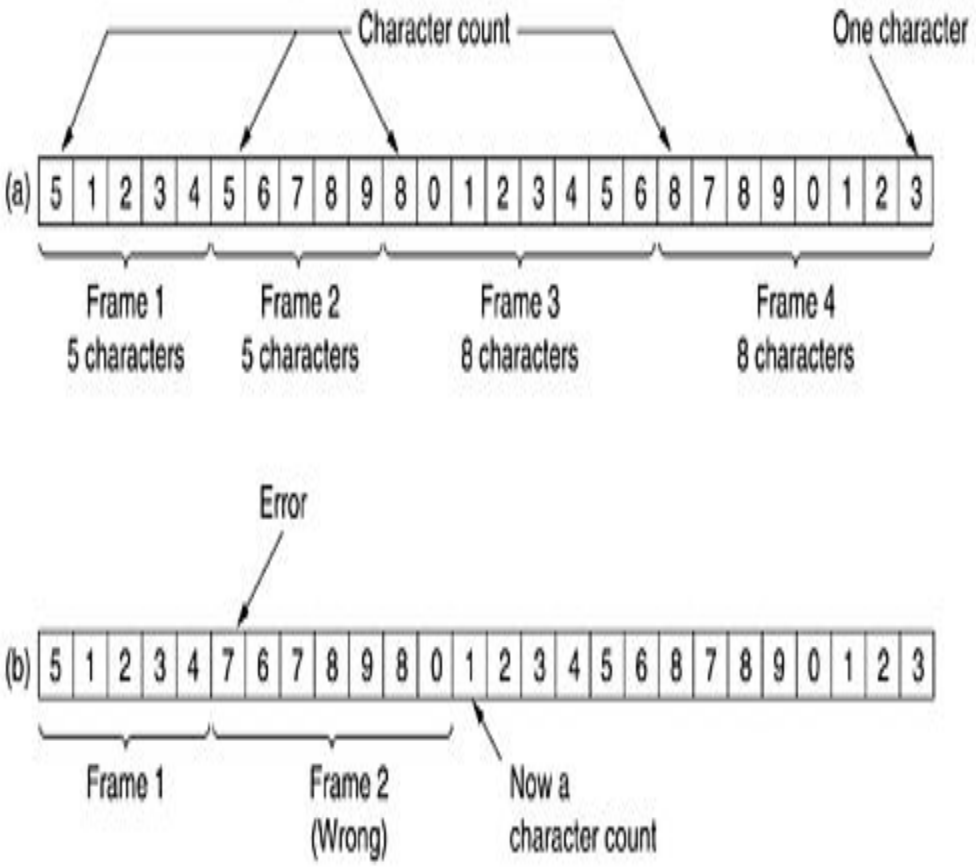
- Receiver can acknowledge frames without permitting further transmission (Receive Not Ready (RNR))
  - ACK works as permission to sender
- Must send a normal acknowledge frame to resume
- If duplex, use **piggybacking**, i.e. load the ACK to carry on top of data frame
  - If no data to send, use acknowledgement frame
  - If data but no acknowledgement to send, send last acknowledgement number again, or have ACK valid flag (TCP)

Framing

- PHY Layer accepts a raw bit stream and attempt to deliver it to the destination with possible bit errors during transmission.
- DLL detects and, if necessary, correct errors, thru breaking the bit stream up into **discrete units called frames** and **compute the checksum** for each frame.
- **Problem is to identify the starting and ending of the frame.**
- Techniques of Framing:
  - Character count.
  - Flag bytes with byte stuffing.
  - Starting and ending flags, with bit stuffing.
  - Physical layer coding violations.

Framing Techniques

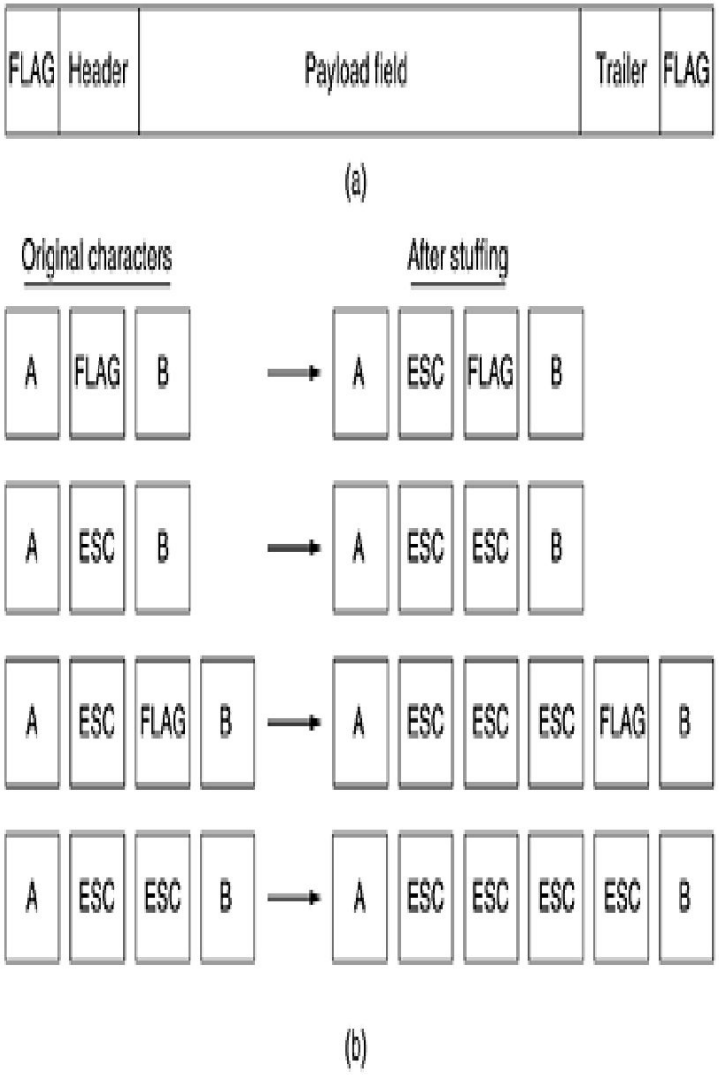
- **Character count**
  - uses a field in the header to specify the number of characters in the frame so that destination knows how many characters follow and hence where the end of the frame is.
  - Disadvantage:
    - the count can be garbled by a transmission error.
    - For example, if the character count of 5 in the second frame becomes a 7, the destination will get out of synchronization and will be unable to locate the start of the next frame.



Framing Techniques

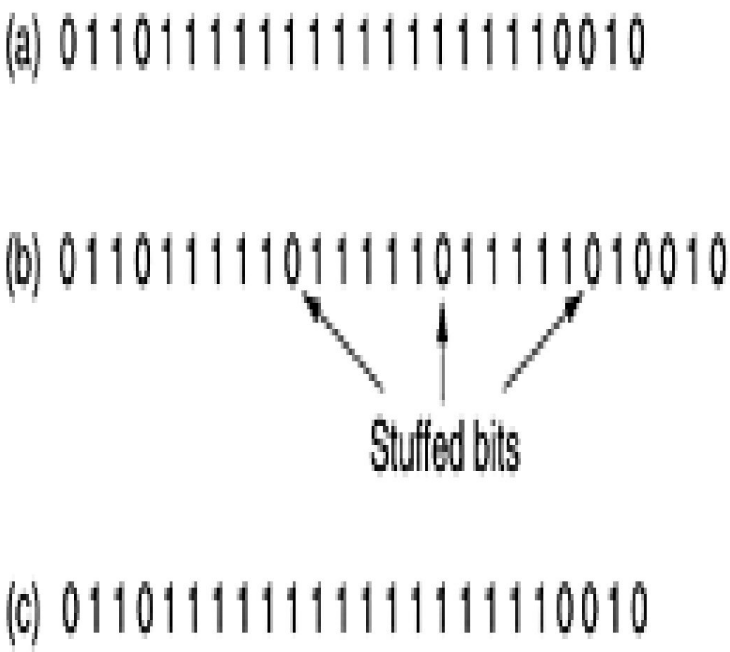
Flag bytes with byte stuffing

- This method gets around the problem of resynchronization after an error by having each frame start and end with special bytes. The starting and ending bytes used are the same byte, called a FLAG byte, as both the starting and ending delimiter.
- Two consecutive flag bytes indicate the end of one frame and start of the next one.
- If binary data are being transmitted, it may easily happen that the flag byte's bit pattern occurs in the data.
  - Solution: sender's DLL inserts a special escape byte (ESC) just before each "accidental" flag byte in the data. The DLL on the receiver removes the escape byte before the data are given to the L3. This technique is **called byte stuffing or character stuffing**. Thus, a framing flag byte can be distinguished from one in the data by the absence or presence of an escape byte before it.
- Disadvantage:
  - It is closely tied to the use of 8-bit characters. Not all character codes use 8-bit characters. For example. UNICODE uses 16-bit characters



Framing Techniques

- **Starting and ending flags, with bit stuffing.**
  - Each frame begins and ends with a special bit pattern, 01111110 (in fact, a flag byte). Whenever the sender's data link layer encounters five consecutive 1s in the data, it automatically stuffs a 0 bit into the outgoing bit stream. This bit stuffing is analogous to byte stuffing, in which an escape byte is stuffed into the outgoing character stream before a flag byte in the data.
  - When the receiver sees five consecutive incoming 1 bits, followed by a 0 bit, it automatically destuffs (i.e., deletes) the 0 bit
  - If the user data contain the flag pattern, 01111110, this flag is transmitted as 011111010 but stored in the receiver's memory as 01111110



Framing Techniques

- Physical layer coding violations.
  - Once the bit-stuffing method is applied during framing, it allows the DLL to use the **L1 encoding violations** to enhance the framing schemes.
  - Scrambling encoding (B8ZS and HDB3 scheme)
    - Uses violations of AMI coding

Error Handling

- Two techniques
  - Error Detection and Correction
    - Done by FEC (ECC) adopting Humming code
    - Or by EDC (CRC) i.e. polynomial code
  - Error Control
    - At protocol level in LLC of DLL

Error Detection and Correction

- Transmission errors on media (e.g., radio) tend to come in bursts rather than singly.
  - burst errors are harder to correct than single errors
- Two basic strategies for dealing with errors.
  - One way is to include enough redundant information along with each block of data sent, to enable the receiver to deduce what the transmitted data must have been. It uses **Error-Correcting Codes.** The use of error-correcting codes is often **referred to as forward error correction**.
  - The other way is to include only enough redundancy to allow the receiver to deduce that an error occurred, but not which error, and have it request a retransmission. It uses **Error-Detecting Codes**.

Error Detection and Correction

- On channels that are highly reliable, such as fiber, it is cheaper **to use an error detecting code** and just retransmit the occasional block found to be faulty.
- On channels such as wireless links that make many errors, it is better to add enough redundancy to each block (**to use an error correcting code)** for the receiver to be able to figure out what the original block was, rather than relying on a retransmission, which itself may be in error.

- Error-Correcting Codes.
  - Refer to discussion in Lecture 2

Error Correcting Code and Error Detecting Code

- Two basic strategies for dealing with errors.
  - One way is to include enough redundant information along with each block of data sent, to enable the receiver to deduce what the transmitted data must have been.
    - This strategy uses error-correcting codes **[Refer Lecture 2]**
    - The use of error-correcting codes is often referred to as forward error correction.
    - Use of Hamming code to correct errors
  - The other way is to include only enough redundancy to allow the receiver to deduce that an error occurred, but not which error, and have it request a retransmission. This strategy uses error-detecting codes.
    - Polynomial code, also known as a CRC (Cyclic Redundancy Check). Polynomial codes are based upon treating bit strings as representations of polynomials with coefficients of 0 and 1 only

Error Detection

- Additional bits are added by transmitter for error detection code
- Parity
  - Simplest error-detecting scheme is to append a parity bit to the end of a block of data
  - Value of parity bit is such that character has even (even parity) or odd (odd parity) number of ones
  - If two of bits are inverted due to error, an undetected errors occur.
  - Typically, even parity is used for synchronous transmission and odd parity for asynchronous transmission.
- One of the most common, and one of the most powerful, error-detecting codes is the cyclic redundancy check (CRC).

Cyclic Redundancy Check

- For a block of bits or message of *k* bits, transmitter generates *n* bit sequence,
  - known as a **frame check sequence (FCS)**, such that the resulting frame, consisting of *k+n* bits, is exactly divisible by some predetermined number
- Transmit *k+n* bits which is exactly divisible by some number
- Receiver divides frame by that number
  - If no remainder, assume no error
  - **For math, see Stallings chapter 7**

Error-detecting Codes

- CRC (polynomial code)(cyclic redundancy check).
- K=bit in the frame.
- Polynomial=x^{k-1} to x^0 [degree k-1].
- Example: frame=110001.

  K= 6.

  Six-term polynomial with coefficients.

  1,1,0,0,0 and 1 : $x^5+x^4+x^0$.
- g(x) is generator polynomial agreed between sender and receiver
- Sender appends checksum to the end of the frame
- Receiver tries dividing it by g(x). If there is a reminder, there has been a transmission error.

Error-detecting Codes

- Error-detecting codes use the polynomial code, also known as a CRC (Cyclic Redundancy Check).
- Polynomial codes are based upon treating bit strings as representations of polynomials with coefficients of 0 and 1.
- Polynomial arithmetic is done modulo 2, according to the rules of algebraic field theory. There are no carries for addition or borrows for subtraction. Both addition and subtraction are identical to Exclusive OR (ExOR).
- For example,

$$\begin{array}{r} 10011011 \\ +\ 11001010 \\ \hline 01010001 \end{array} \qquad \begin{array}{r} 00110011 \\ +\ 11001101 \\ \hline 11111110 \end{array} \qquad \begin{array}{r} 11110000 \\ -\ 10100110 \\ \hline 01010110 \end{array} \qquad \begin{array}{r} 01010101 \\ -\ 10101111 \\ \hline 11111010 \end{array}$$

- Long division is carried out the same way as it is in binary except that the subtraction is done modulo 2, as above. A divisor is said "to go into" a dividend if the dividend has as many bits as the divisor.

Error-detecting Codes

- CRC (polynomial code)(cyclic redundancy check).
  - K=bit in the frame.
  - Polynomial=x^{k-1} to x^0 [degree k-1].
- Example: frame=110001.

  K= 6.

  Six-term polynomial with coefficients.

  1,1,0,0,0 and 1:$x^5+x^4+x^0$.
- IEEE 802 uses the standard polynomial i.e.

$$x^{32}+x^{26}+x^{23}+x^{22}+x^{16}+x^{12}+x^{11}+x^{10}+x^{8}+x^{7}+x^{5}+x^{4}+x^{2}+x^{1}+1$$

Error-detecting Codes

- When the polynomial code method is employed, the sender and receiver must agree upon a **generator polynomial, G(x),** in advance.
- Both the high- and low-order bits of the generator must be 1.
- To compute the checksum for some frame with m bits, corresponding to the polynomial M(x), the frame must be longer than the generator polynomial.
- The **idea is to append a checksum to the end of the frame** in such a way that the polynomial represented by the checksummed frame is divisible by G(x). When the receiver gets the checksummed frame, it tries dividing it by G(x). If there is a remainder, there has been a transmission error.

The algorithm

- The algorithm for computing the checksum is as follows:

1. Let r be the degree of G(x). Append r zero bits to the low-order end of the frame so it now contains m+r bits and corresponds to the polynomial $x^r M(x)$.

2. Divide the bit string corresponding to G(x) into the bit string corresponding to $x^r M(x)$, using **modulo 2 division.**

3. Subtract the remainder (which is always r or fewer bits) from the bit string corresponding to $x^r M(x)$ using **modulo 2 subtraction**. The result is the checksummed frame to be transmitted. Call its polynomial T(x).

Example

- **Problem:** Calculate the polynomial code checksum for a frame 1101011011 using the generator $G(x) = x^4 + x + 1$
- T(x) will be

```
   11010110110000
 -          1110
 ----------------
   11010110111110
```

- Upon receiving the checksummed frame, the receiver divides it by G(x); that is, it computes **[T(x) + E(x)]/G(x)**
- Where, E(x) is the possible errors during tansmission.

Frame : 1 1 0 1 0 1 1 0 1 1

Generator: 1 0 0 1 1

Message after 4 zero bits are appended: 1 1 0 1 0 1 1 0 1 1 0 0 0 0

```
                1 1 0 0 0 0 1 0 1 0
10011 | 1 1 0 1 0 1 1 0 1 1 0 0 0 0
        1 0 0 1 1
          1 0 0 1 1
          1 0 0 1 1
            0 0 0 0 1
            0 0 0 0 0
              0 0 0 1 0
              0 0 0 0 0
                0 0 1 0 1
                0 0 0 0 0
                  0 1 0 1 1
                  0 0 0 0 0
                    1 0 1 1 0
                    1 0 0 1 1
                      0 1 0 1 0
                      0 0 0 0 0
                        1 0 1 0 0
                        1 0 0 1 1
                          0 1 1 1 0
                          0 0 0 0 0
                            1 1 1 0
```

Remainder

T(x)

Transmitted frame: 1 1 0 1 0 1 1 0 1 1 1 1 0

Error-detecting Codes

- What kinds of errors will be detected? i.e. the term E(x)?
  - If T(x)/G(x) is 0, the result of the computation is simply E(x)/G(x). i.e. those errors that happen to correspond to polynomials containing G(x) as a factor will slip by; all other errors will be caught.
  - **If there has been a single-bit error, $E(x) = x^i$**, where i determines which bit is in error. If G(x) contains two or more terms, it will never divide E(x), so all single-bit errors will be detected.
  - **A burst error of length k can be represented by $x^i (x^{k-1} + \ldots + 1)$**, where i determines how far from the right-hand end of the received frame the burst is located. If G(x) contains an x^0 term, it will not have x^i as a factor, so if the degree of the parenthesized expression is less than the degree of G(x), the remainder can never be zero.

Error Control

- Detection and correction of errors
- Lost frames
- Damaged frames
- **Automatic repeat request (ARQ)**
  - Error detection
  - Positive acknowledgment
    - Acknowledgment to successfully received, error-free frames.
  - Retransmission after timeout
  - Negative acknowledgement and retransmission
    - Acknowledgment to frames in which an error is detected. The source retransmits such frames.
- Collectively, these mechanisms are all referred to as **Automatic Repeat reQuest** (ARQ); the effect of ARQ is to turn an unreliable data link into a reliable one.

Automatic Repeat ReQuest (ARQ)

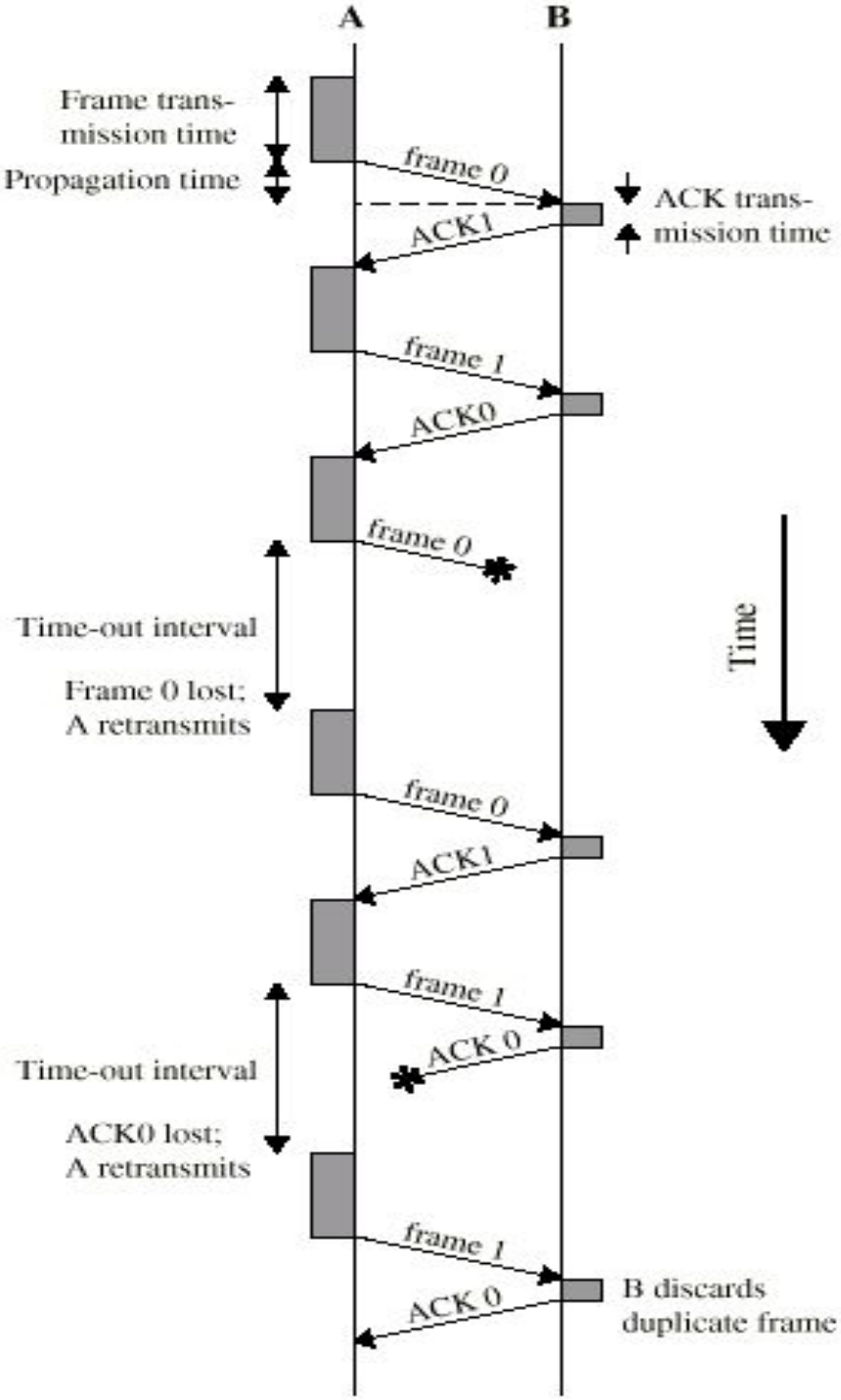
Three versions of ARQ have been standardized:

- Stop and wait
- Go back N
- Selective reject (selective repeat/retransmission)

Stop and Wait

- Source transmits single frame
- Wait for ACK
- If received frame is damaged, discard it
  - Transmitter has **timeout**
  - If no ACK within timeout, retransmit
- If ACK is damaged, transmitter will not recognize it
  - Transmitter will **retransmit**
  - Receiver gets two copies of frame
  - Use ACK0 and ACK1

Stop and Wait - Diagram



Stop and Wait - Pros and Cons

- Simple
  - Works well for single message sequences
- Inefficient line use, because
  - With the use of multiple frames for a single message, the stop-and-wait procedure may be inadequate as it allows only one frame at a time that can be in transit.
  - In situations where the **bit length of the link is greater than the frame length** serious inefficiencies result.

Enhancement:

- The sliding-window flow control technique can be adapted to provide more efficient line use; in this context, it is sometimes referred to as *continuous ARQ*.

Go Back N

- Based on sliding window
- If no error, ACK as usual with next frame expected
- Use window to control number of outstanding frames
- If error, reply with rejection
  - Discard that frame and all future frames until error frame is received correctly
  - Transmitter must go back and retransmit that frame and all subsequent frames

Go Back N - Damaged Frame

- <u>Situation:</u> Receiver detects error in frame *i*
- Receiver sends rejection-*i* *(i.e. REJ i)*
- Transmitter gets rejection-*i*
- Transmitter retransmits frame *i* and all subsequent frames

Go Back N - Lost Frame (1)

- <u>Situation:</u> Frame *i* is lost
- Transmitter sends *i+1*
- Receiver gets frame *i+1* out of sequence
- Receiver sends reject *i*
- Transmitter goes back to frame *i* and retransmits

Go Back N - Lost Frame (2)

- <u>Situation:</u> Frame *i* lost and no additional frame is sent
- Receiver gets nothing and returns neither acknowledgement nor rejection
- Transmitter times out and sends acknowledgement frame with **P bit set to 1**
- **P stands for Poll**
- Receiver interprets this as **command** which it acknowledges with the number of the next frame it expects (frame *i*)
- Transmitter then retransmits frame *i*

Go Back N - Damaged Acknowledgement

- <u>Situation:</u> Receiver gets frame *i* and send acknowledgement (*i+1*) which is lost
- Acknowledgements are **cumulative**, so next acknowledgement (*i+n*) may arrive before transmitter times out on frame *i*
- If transmitter times out, it sends acknowledgement with P bit set as before
- This can be repeated a number of times before a reset procedure is initiated

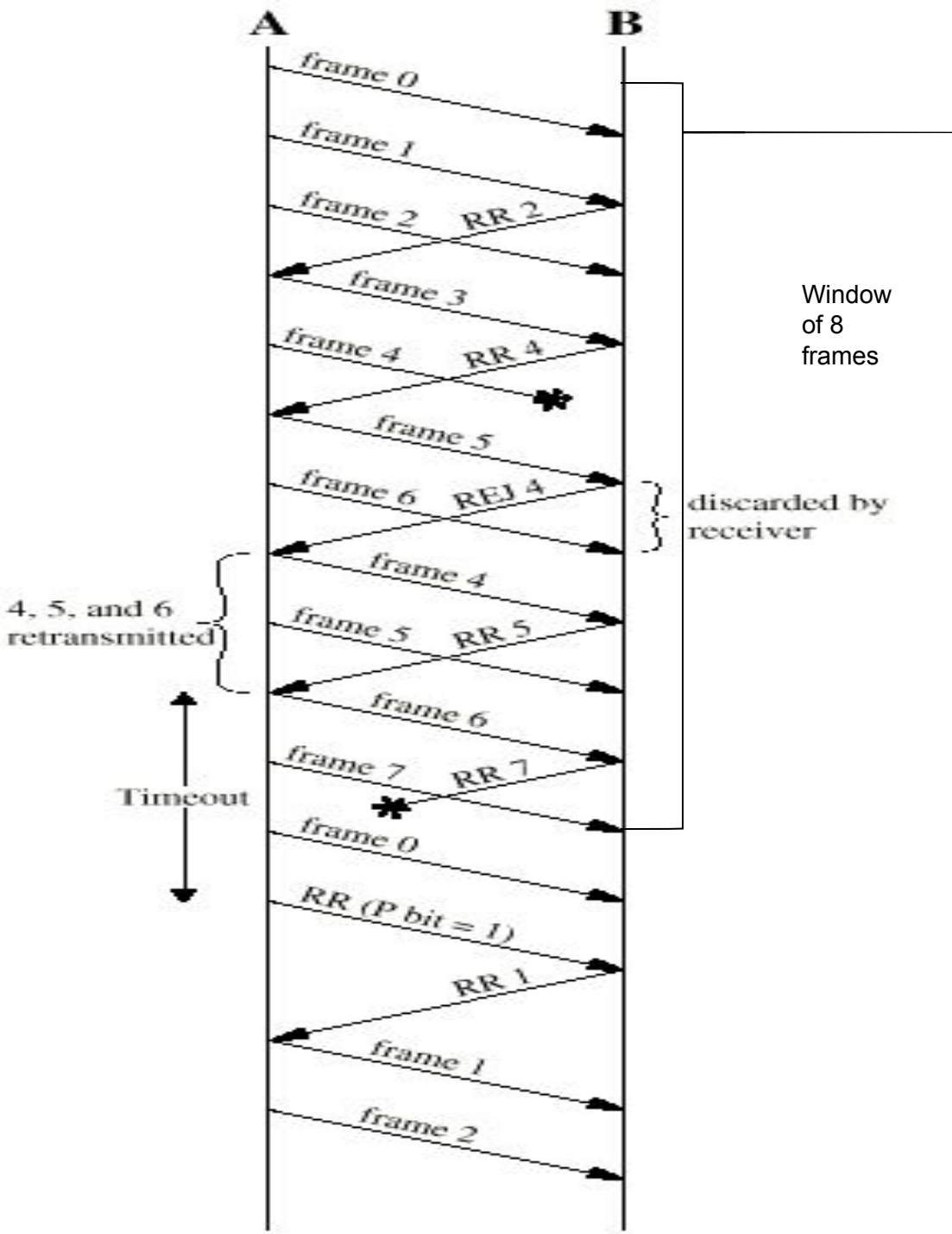
Go Back N - Damaged Rejection

- As for lost frame (2)
  - uses P bit to pull the receiver

Go Back N - Diagram

RR : Receive Ready or piggybacked ACK

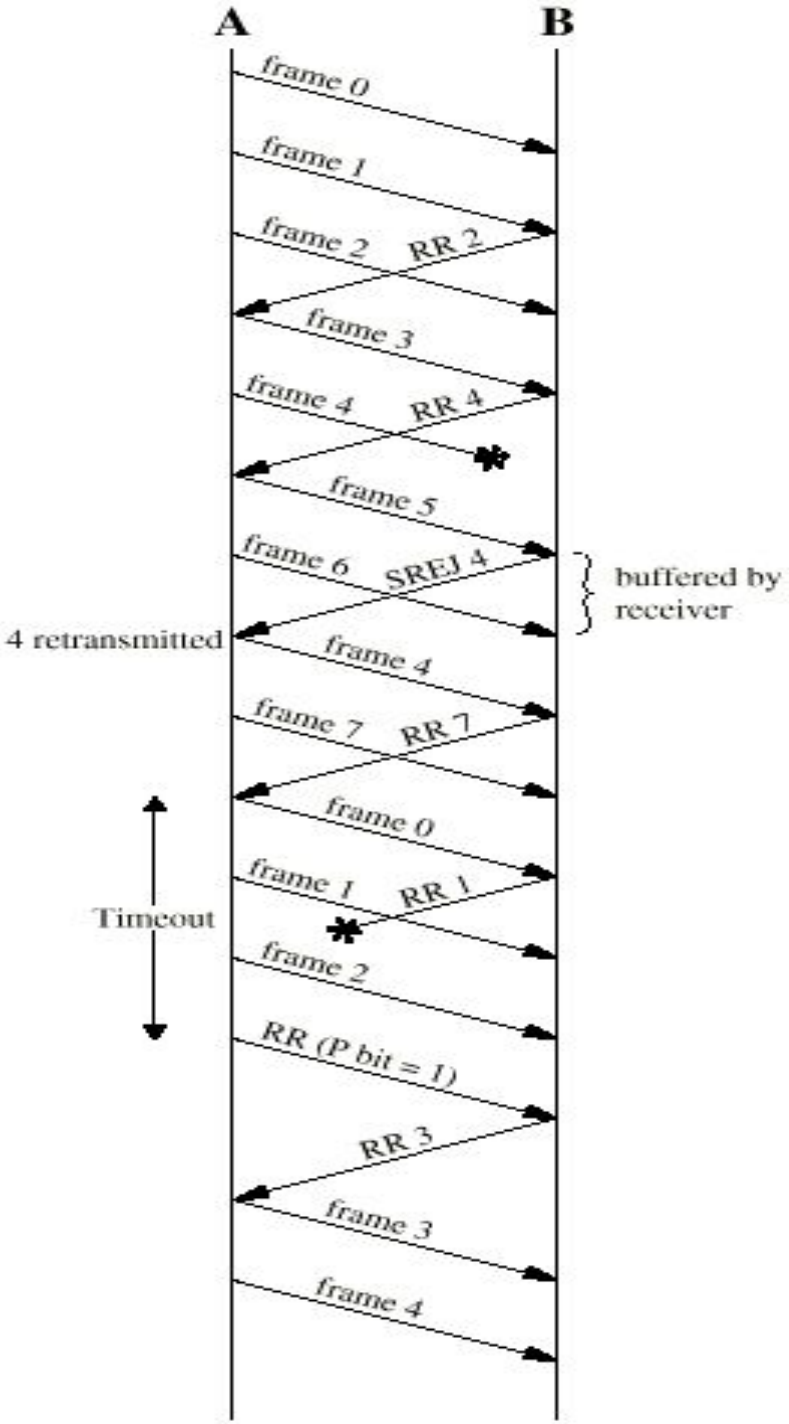
REJ: Rejection



Selective Reject

- Selective reject would appear to be more efficient than go-back-N, because it minimizes the amount of retransmission
- Also called **selective retransmission**
- Only rejected frames are retransmitted
- Subsequent frames are accepted by the receiver and **buffered**
- Minimizes retransmission
- **Receiver must maintain large enough buffer** and implements logic for reinserting rejected frame in the proper sequence .
- More complex logic in transmitter, to be able to send a frame out of sequence.

Selective Reject - Diagram



Example Data Link Protocols

- HDLC
  - is a classical **bit-oriented protocol** whose variants have been in wide use for decades in many applications.
- PPP
  - is the data link protocol used to connect home computers to the Internet.
- Other DLC Protocols
  - LAPB
  - LAPD
  - LLC
  - Frame Relay ☐ LAPF
  - ATM ☐ Not HDLC based

High Level Data Link Control

- Bit oriented protocol
- ISO 33009, ISO 4335
- History:
  - First used in the IBM mainframe world: SDLC (Synchronous Data Link Control) protocol. After developing SDLC, IBM submitted it to ANSI and ISO for acceptance as U.S. and international standards, respectively. ANSI modified it to become ADCCP (Advanced Data Communication Control Procedure), and ISO modified it to become HDLC (High-level Data Link Control). CCITT(Consultative Committee for International Telephony and Telegraphy) then adopted and modified HDLC for its LAP (Link Access Procedure) as part of the X.25 network interface standard but later modified it again to LAPB, to make it more compatible with a later version of HDLC.

Basic Characteristics

- HDLC defines
  - three types of stations,
  - two link configurations, and
  - three data transfer modes of operation.

HDLC Station Types

- Primary station
  - Controls operation of link
  - Frames issued are called **commands**
  - Maintains separate logical link to each secondary station
- Secondary station
  - Under control of primary station
  - Frames issued called **responses**
- Combined station
  - Combines the features of primary and secondary
  - A combined station may issue commands and responses

HDLC Link Configurations

- Unbalanced
  - Consists of one primary and one or more secondary stations
  - Supports full duplex and half duplex
- Balanced
  - Consists of two combined stations
  - Supports full duplex and half duplex

HDLC Transfer Modes (1)

- Normal Response Mode (NRM)
  - Used with an Unbalanced configuration
  - Primary initiates transfer to secondary
  - Secondary may only transmit data in response to command from primary
  - Used on multi-drop lines
  - Host computer as primary
  - Terminals as secondary

HDLC Transfer Modes (2)

- Asynchronous Balanced Mode (ABM)
  - Used with balanced configuration
  - Either combined station may initiate transmission without receiving permission from the other combined station
  - Most widely used
  - No polling overhead
  - No concept of primary and secondary station

HDLC Transfer Modes (3)

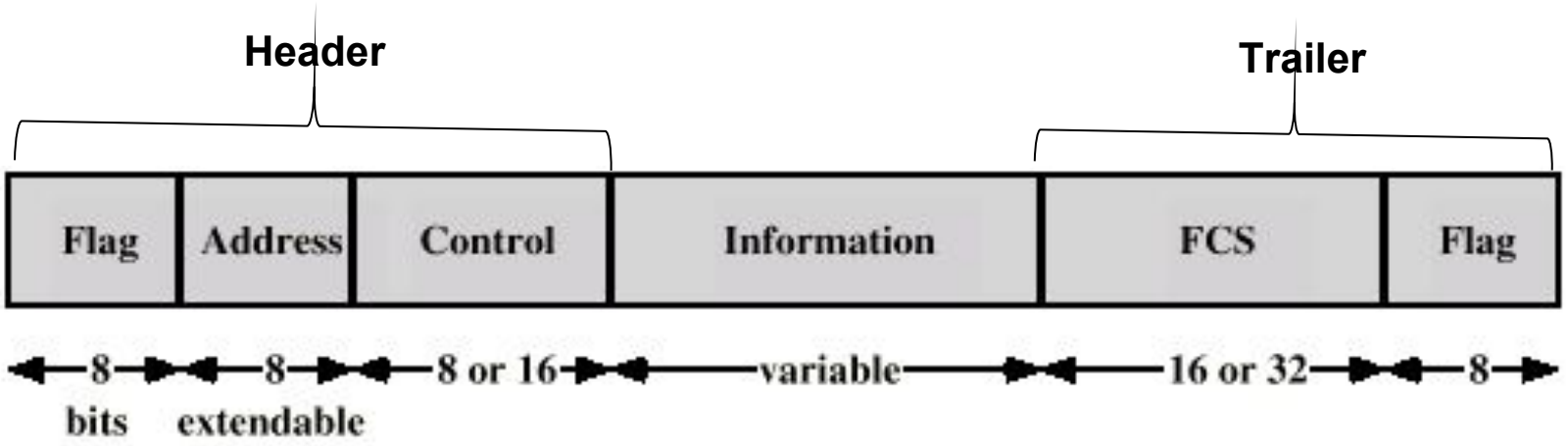
- Asynchronous Response Mode (ARM)
  - Used with unbalanced configuration
  - Secondary may initiate transmission without permission from primary
  - The primary still retains responsibility for the line, including initialization, error recovery, and logical disconnection.
  - rarely used

Frame Structure

- HDLC uses Synchronous transmission
- All transmissions are in the form of frames, and
- Single frame format is used for all types of data and control exchanges derived from standard bit-oriented protocol format as follows:

Bits	8	8	8	≥ 0	16	8
	0 1 1 1 1 1 1 0	Address	Control	Data	Checksum	0 1 1 1 1 1 1 0

Frame Structure Diagram



Flag Fields

- Delimit frame at both ends with a unique pattern 01111110
- May close one frame and open another
- Receiver continues hunts for flag sequence to synchronize
- **Bit stuffing** is used to avoid confusion with data containing 01111110
  - 0 inserted after every sequence of five 1s
  - When a pattern of five 1s appears, the sixth bit is examined by the receiver.
    - If it is 0, it is deleted
    - If it is 1 and seventh bit is 0, accept as flag
    - If sixth and seventh bits are 1, sender is indicating **abort**

Bit Stuffing

- The presence of arbitrary bit patterns (i.e., there are no restrictions on the content of the various fields imposed by the link protocol), **there is no assurance that the pattern 01111110 will not appear somewhere inside the frame**, **thus destroying synchronization**. To avoid this problem, a procedure known as *bit stuffing* is used.

- For all bits between the starting and ending flags, the transmitter inserts an extra 0 bit after each occurrence of five 1s in the frame, is called the bit stuffing.

- With the use of bit stuffing, arbitrary bit patterns can be inserted into the data field of the frame. This property is known as **data transparency**.

Bit Stuffing

- Example with possible errors

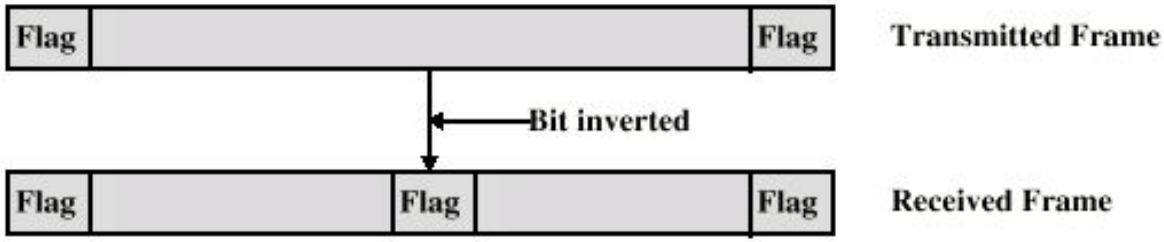
Original Pattern:

111111111111011111101111110

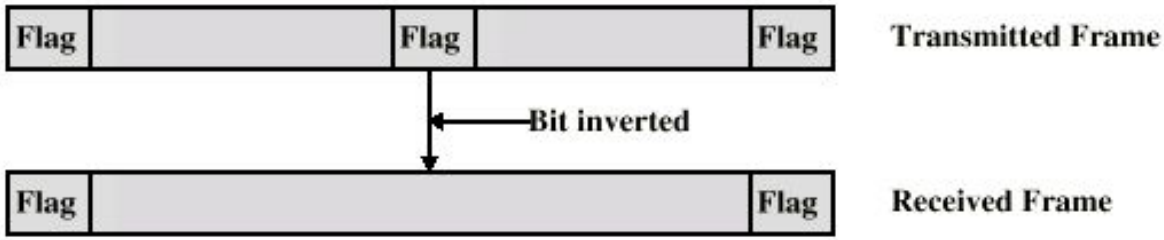
After bit-stuffing

111110111111011011111010111111010

(a) Example



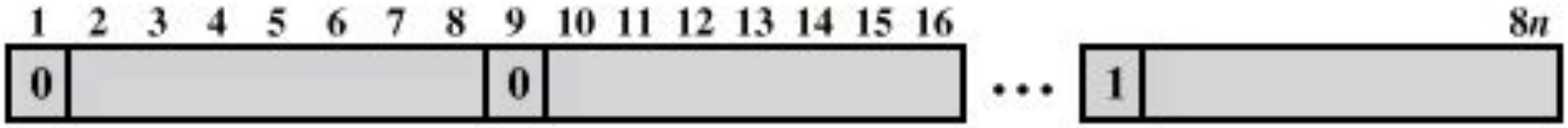
(b) An inverted bit splits a frame in two



(c) An inverted bit merges two frames

Address Field

- Identifies secondary station that sent or will receive frame
- Usually 8 bits long
- May be extended to multiples of 7 bits
  - LSB of each octet indicates that it is the last octet (1) or not (0)
- All ones (11111111) is broadcast

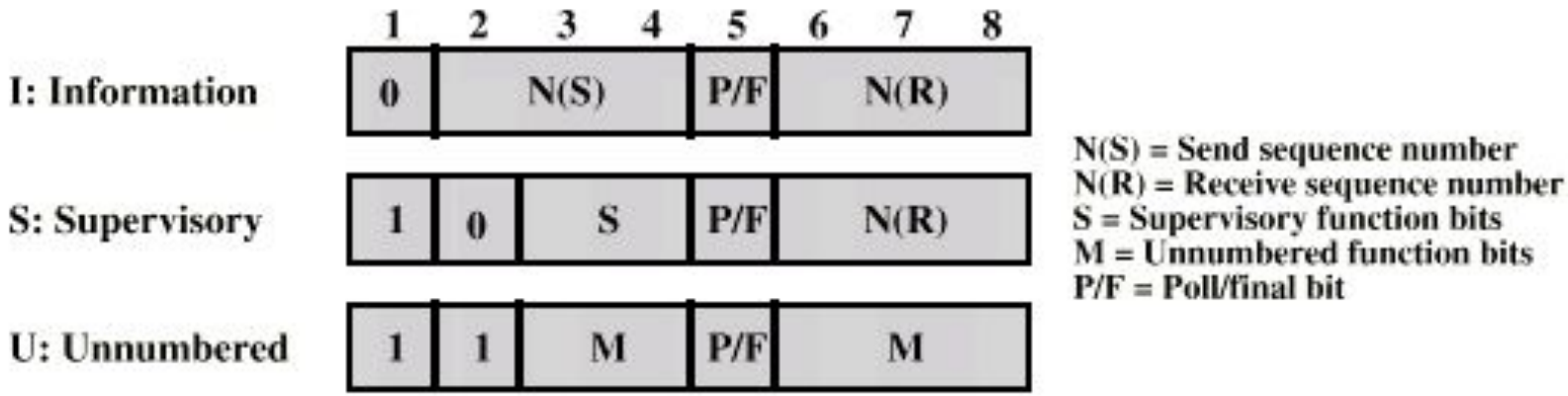


(b) Extended Address Field

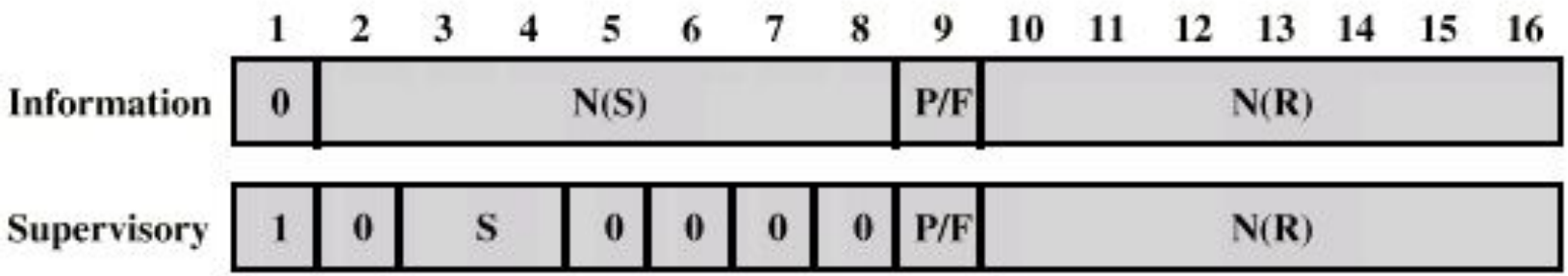
Control Field

- Different for different frame type
  - **Information** - data to be transmitted to user (next layer up)
    - Flow and error control piggybacked on information frames
  - **Supervisory** - ARQ when piggyback not used
  - **Unnumbered** - supplementary link control
- First one or two bits of control filed identify frame type
- Remaining bits explained later

Control Field Diagram



(c) 8-bit control field format



(d) 16-bit control field format

Poll/Final Bit

- Use depends on context
- Command frame
  - P bit
  - 1 to solicit (poll) response from peer
- Response frame
  - F bit
  - 1 indicates response to soliciting command

Information Field

- Only in information and some unnumbered frames
- Must contain integral number of octets
- Variable length

Frame Check Sequence Field

- FCS
- Error detection
- 16 bit CRC
- Optional 32 bit CRC

HDLC Operation

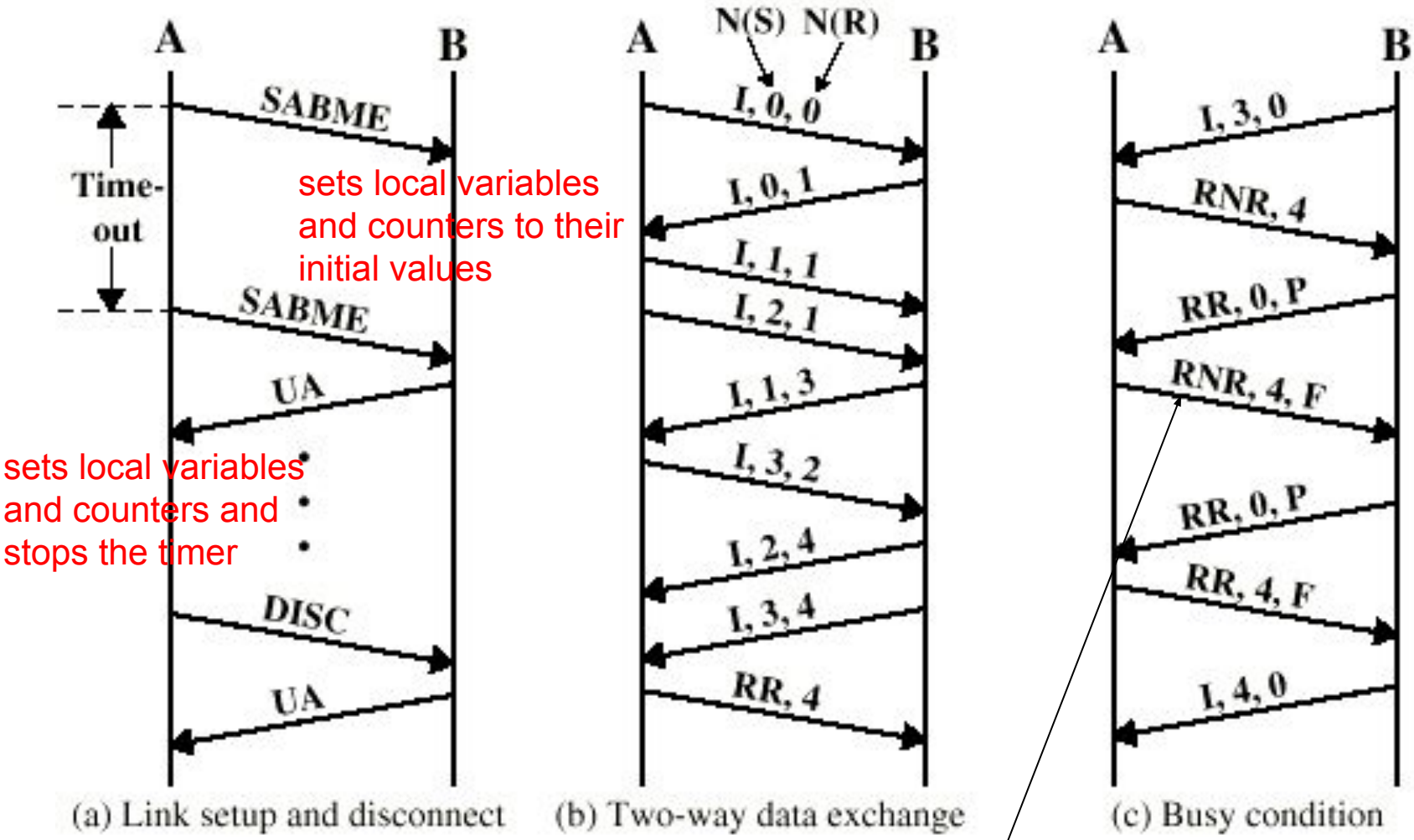
- Exchange of information, supervisory and unnumbered frames
- Three phases
  - Initialization
  - Data transfer
  - Disconnect

HDLC Commands and Responses

Table 7.1 HDLC Commands and Responses

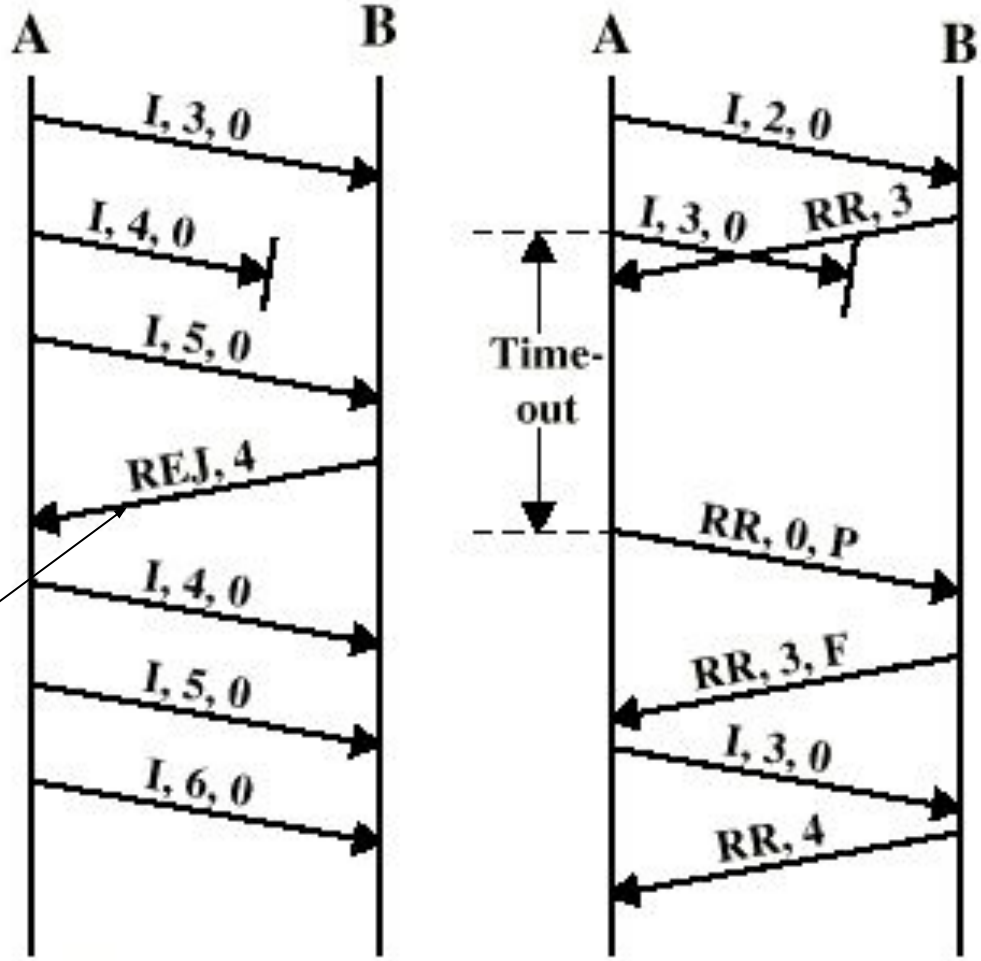
Name	Command/ Response	Description
Information (I)	C/R	Exchange user data
Supervisory (S)		
Receive ready (RR)	C/R	Positive acknowledgment; ready to receive I-frame
Receive not ready (RNR)	C/R	Positive acknowledgment; not ready to receive
Reject (REJ)	C/R	Negative acknowledgment; go back N
Selective reject (SREJ)	C/R	Negative acknowledgment; selective reject
Unnumbered (U)		
Set normal response/extended mode (SNRM/SNRME)	C	Set mode; extended = 7-bit sequence numbers
Set asynchronous response/extended mode (SARM/SARME)	C	Set mode; extended = 7-bit sequence numbers
Set asynchronous balanced/extended mode (SABM, SABME)	C	Set mode; extended = 7-bit sequence numbers
Set initialization mode (SIM)	C	Initialize link control functions in addressed station
Disconnect (DISC)	C	Terminate logical link connection
Unnumbered Acknowledgment (UA)	R	Acknowledge acceptance of one of the set-mode commands
Disconnected mode (DM)	R	Responder is in disconnected mode
Request disconnect (RD)	R	Request for DISC command
Request initialization mode (RIM)	R	Initialization needed; request for SIM command
Unnumbered information (UI)	C/R	Used to exchange control information
Unnumbered poll (UP)	C	Used to solicit control information
Reset (RSET)	C	Used for recovery; resets N(R), N(S)
Exchange identification (XID)	C/R	Used to request/report status
Test (TEST)	C/R	Exchange identical information fields for testing
Frame reject (FRMR)	R	Report receipt of unacceptable frame

Examples of Operation (1)



RNR asks to suspend transmission of I-frames with ACKing

Examples of Operation (2)



(d) Reject recovery (e) Timeout recovery

- Initiates go-back N ARQ
- SREJ is sent to initiate Selective Reject/Repeat

PPP (Point-to-Point Protocol)

- PPP is defined in RFC 1661 (further elaborated in RFCs 1662 and 1663).
- **Point-to-Point Protocol** (**PPP**) is a DLL protocol commonly used in establishing a **direct connection between two nodes**.
- PPP functions
  - PPP handles error detection, supports multiple protocols, allows IP addresses to be negotiated at connection time, permits authentication, and has many other features.
- PPP provides three features:
  - A **framing** method that unambiguously delineates the end of one frame and the start of the next one. The frame format also handles error detection.
  - A link control protocol for bringing lines up, testing them, negotiating options, and bringing them down again gracefully when they are no longer needed. This protocol is called **LCP (Link Control Protocol)**. It supports synchronous and asynchronous circuits and byte-oriented and bit-oriented encodings.
  - A way to negotiate network-layer options in a way that is independent of the network layer protocol to be used. The method chosen is to have a different **NCP (Network Control Protocol)** for each network layer supported.

PPP (Point-to-Point Protocol)

- The PPP frame format was chosen to closely resemble the HDLC frame format.
- The major difference between PPP and HDLC is that **PPP is character oriented rather than bit oriented**. PPP **uses byte stuffing on dial-up modem** lines, so all frames are an integral number of bytes.
- It is not possible to send a frame consisting of 30.25 bytes, as it is with HDLC. Not only can PPP frames be sent over dial-up telephone lines, but they can also be sent over SONET or true bit-oriented HDLC lines.
- PPP frame format Unnumbered Mode operation

Bytes	1	1	1	1 or 2	Variable	2 or 4	1
	Flag 01111110	Address 11111111	Control 00000011	Protocol	Payload	Checksum	Flag 01111110

Point-to-Point Protocol (PPP)

- **Point-to-Point Protocol** (**PPP**) is a data link) is a data link protocol commonly used in establishing a direct connection between two nodes.
- It can provide connection authenticationIt can provide connection authentication, transmission encryptionIt can provide connection authentication, transmission encryption (using ECPIt can provide connection authentication, transmission encryption (using ECP, RFC 1968It can provide connection authentication, transmission encryption (using ECP, RFC 1968), and compression.
- Two encapsulated forms of PPP,
  - Point-to-Point Protocol over Ethernet (PPPoE) and
  - Point-to-Point Protocol over ATMPoint-to-Point Protocol over ATM (PPPoA), are used most commonly by Internet Service Providers (ISPs) to establish a Digital Subscriber Line (DSL) Internet service connection with customers.
- PPP is commonly used as a data link layer protocol for connection over synchronousPPP is commonly used as a data link layer protocol for connection over synchronous and asynchronous circuitsPPP is commonly used as a data link layer protocol for connection over synchronous and asynchronous circuits, where it has largely

Point-to-Point Protocol

- PPP is used over many types of physical networks including serial cablePPP is used over many types of physical networks including serial cable, phone linePPP is used over many types of physical networks including serial cable, phone line, trunk linePPP is used over many types of physical networks including serial cable, phone line, trunk line, cellular telephone, specialized radio links, and fiber optic links such as SONET (Synchronous Optical NETworking).
- PPP is also used over Internet access connections (now marketed as "broadband").
- Internet service providersInternet service providers (ISPs) have used PPP for customer dial-up accessInternet service providers (ISPs) have used PPP for customer dial-up access to the InternetInternet

Other DLC Protocols

- DLC Protocols
  - LAPB
  - LAPD
  - LLC
  - Frame Relay ☐ LAPF
  - ATM ☐ Not HDLC based

Other DLC Protocols (LAPB,LAPD)

- Link Access Procedure, Balanced (LAPB)
  - Part of X.25 (ITU-T)
  - Subset of HDLC - ABM
  - Point to point link between system and packet switching network node
- Link Access Procedure, D-Channel
  - ISDN (ITU-D)
  - ABM
  - Always 7-bit sequence numbers (no 3-bit)
  - 16 bit address field contains two sub-addresses
    - One for device and one for user (next layer up)

Other DLC Protocols (LLC)

- Logical Link Control (LLC)
  - IEEE 802 standard
  - Different frame format
  - Link control split between medium access layer (MAC) and LLC (on top of MAC)
  - No primary and secondary - all stations are peers
  - Two addresses needed
    - Sender and receiver
  - Error detection at MAC layer
    - 32 bit CRC
  - Destination and source access points (DSAP, SSAP)

Other DLC Protocols (Frame Relay) (1)

- **Streamlined capability over high speed packet switched networks**
- Used in place of X.25
- Uses Link Access Procedure for Frame-Mode Bearer Services (LAPF)
- Two protocols
  - Control - similar to HDLC
  - Core - subset of control

Other DLC Protocols (Frame Relay) (2)

- ABM
- 7-bit sequence numbers
- 16 bit CRC
- 2, 3 or 4 octet address field
  - Data link connection identifier (DLCI)
  - Identifies logical connection

- More on frame relay later

Other DLC Protocols (ATM)

- Asynchronous Transfer Mode
- **Streamlined capability across high speed networks**
- **Not HDLC based**
- Frame format called “cell”
- Fixed 53 octet (424 bit)

- Details later

Required Reading

- Stallings chapter 7
- Tanenbaum chapter 3
- Web sites on HDLC, frame relay, Ethernet and ATM, PPP